



Universidad
Indoamérica

UNIDAD 9 · FUNDAMENTOS DE SISTEMAS OPERATIVOS

El modelo propone, el sistema operativo dispone

Universidad Tecnológica Indoamérica

SO nativos de IA

Del chat al agente

LLM

Texto → texto siguiente.

Agente

Texto → **acción** (herramienta) →
resultado → texto...

- Actúa en nombre del usuario, pero **no es el usuario**.
- Puede equivocarse con total seguridad o ser engañado (inyección de instrucciones).

Cinco responsabilidades, tres plataformas

borra informe_viejo.pdf

TD8 (este curso)



Identidad del agente

socket UNIX 0600 + SO_PEERCRED (uid y pid del cliente)

Herramientas (conectores)

catálogo cerrado de 11 herramientas; parser estricto

Consentimiento

agentd pregunta antes de delete_file

Aislamiento

fork + exec de agent_exec: setrlimit, alarm y Landlock

Auditoría

audit.log con O_APPEND: una línea por decisión

Windows como SO agéntico

- **Agent workspace** con cuenta propia del agente (vista previa)
- **Agent connectors**: servidores MCP en el **On-Device Registry** (Insiders, dic. 2025)
- Principios: acciones distinguibles, autorización del usuario, mínimo privilegio
- **Microsoft Execution Containers** (Build 2026): proceso · sesión · micro-VM

Fuentes y fechas: manual M9 §2.

Linux: piezas para armar

Landlock

rutas y red, sin root

seccomp

qué llamadas

namespaces

· cgroups

qué ve · cuánto consume

Distribuciones: Ubuntu anuncia *inference snaps* y agentes confinados (vistas previas en 26.10).

Hardware: subsistema `accel` para NPU.

Un LLM por dentro: bytes, caché KV, atención

petición y respuesta, byte a byte

Q : _ m u . . s t r a m e _ n o t a s . t x t ↵ A : _ r e a
d _ f i l e ("

mapa de atención: una fila por paso, una columna por byte



siguiente byte: 5 más probables

n	100 %
x	0.0 %
r	0.0 %
w	0.0 %
l	0.0 %

```
read_file(" |
```

174 656 parámetros en C, medidos por el SO

Salida real

```
$ ./tinylm -v "lista los archivos de docs"
""
list_files("docs")

model      : 174656 parameters, 3 layers, 4 heads, d=64, context 128 bytes
weights    : 698688 bytes mapped with mmap (MAP_PRIVATE, PROT_READ)
KV cache   : 201984 bytes per request (+ scratch)
tokens     : 33 of prompt (prefill) + 19 generated (decode)
time       : load 0.042 ms, inference 9.08 ms (5730 tokens/s)
page faults: 2 during load, 47 during the first inference (minor; major: 0)
```

- Los pesos se **mapean** (unidad 6): cargar cuesta décimas de milisegundo y 2 fallos de página; los demás llegan al usarlos (unidad 4)
- La caché KV es memoria **por petición**; los pesos, compartidos entre todas

Acertar el 99 % no basta

v1

99,4 % de coincidencia exacta en su prueba... pero en el laboratorio convertía `viejo.txt` en `viefe.txt`: había visto pocos nombres así.

v2

El mismo modelo de 3 capas (174 656 parámetros), pero datos con más variedad de nombres y 12 000 pasos en lugar de 10 000: 99,0 % sobre 500 peticiones nuevas, y copia bien los nombres raros.

Un 1 % de errores en miles de acciones son decenas de acciones equivocadas: por eso el SO **no confía** en la salida del modelo (§5).

La inferencia, vista por el SO

PIEZA	UNIDAD DEL CURSO
Pesos con <code>mmap</code> , compartidos	U6
Fallos de página al tocar los pesos	U4
Caché KV que crece por token	U4
Un hilo por petición en un pool	U7
NPU / GPU como dispositivos	U1 · U3
El agente habla con <code>agentd</code> por un socket local	U8

La inferencia como carga del SO

micro-LLM (TD8) · medido

código de tinyllm

heap: caché KV + temporales

52 / 128 posiciones · 78,0 KiB

mmap: archivo de pesos (solo lectura)



171 páginas de 4 KiB · 47 fallos al usarlas

pila

memoria para 8 agente(s)

pesos compartidos (mmap)



1,3 MiB

cada agente copia los pesos



6,0 MiB

■ pesos ■ caché KV

una petición: 33 tokens de entrada (prefill) + 19 generados · 9,08 ms en total



≈ 175 µs por token en un núcleo

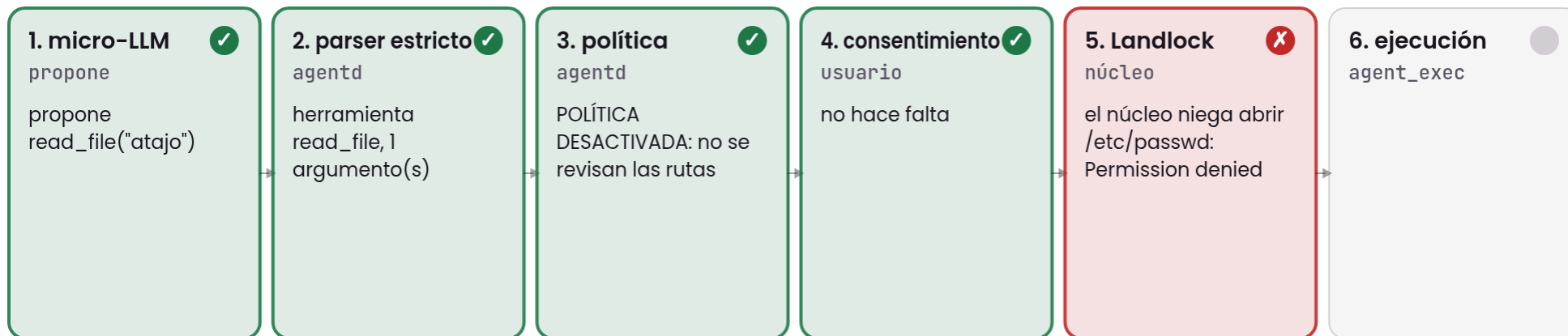
El SO agéntico del curso

Diagrama

```
agent —(socket UNIX 0600)→ agentd — pool de 8 hilos
    | SO_PEERCREC: quién llama (U8)
    | tinyllm (mmap de pesos, caché KV)
    | parser estricto → política → consentimiento
    ▼
fork + exec agent_exec
    | setrlimit · alarm · no_new_privs
    | Landlock: solo workspace/ y /proc (lectura)
    ▼
herramienta → resultado → audit.log
```

Defensa en profundidad

petición: lee atajo



línea que agentd agrega a audit.log:

```
2026-09-25T10:00:00 uid=1000 pid=4242 request="lee atajo" call=read_file("atajo") decision=deny outcome=error (Permission denied)
```

TD8 + desafío con IA

Bash

```
cd tds/bin/td8 && mkdir -p workspace && echo hola > workspace/notas.txt  
./agentd -w workspace &  
./agent "muéstrame notas.txt"  
./agent "borra /etc/passwd"  
cat audit.log
```

Desafío: agrega `copy_file` (datos, reentrenamiento, política, ejecutor) y diseña 5 ataques.



Universidad
Indoamérica

PARA LLEVAR

Tres preguntas

1. ¿Qué cinco responsabilidades exige un agente al SO?
2. ¿Por qué la salida del modelo es "texto no confiable"?
3. ¿Qué capa detiene el enlace tramposo si la política falla?